

TechHub — DevOps, Docker & Kubernetes Governance

Document: 02 — DevOps, Docker & Kubernetes **Version:** 1.0.0 **Owner:** Member 3 (SRE / Platform Lead) **Status:** Active

Table of Contents

1. [DevOps Governance Model](#)
 2. [Shared Responsibility Matrix](#)
 3. [Docker Standards](#)
 4. [Kubernetes Standards](#)
 5. [Kubernetes Ownership Matrix](#)
 6. [Kubernetes Folder Structure](#)
 7. [Docker Compose Integration Plan](#)
 8. [Kubernetes Progressive Deployment Plan](#)
 9. [Environment Variable Standards](#)
 10. [Service Naming Conventions](#)
 11. [Local Development Strategy](#)
 12. [Deployment Workflow](#)
 13. [Redis Caching Standards](#)
 14. [Health Check Standards](#)
-

1. DevOps Governance Model

1.1 Model: Shared Ownership + Central SRE Coordination

TechHub uses a **federated DevOps model**. Every engineer owns the DevOps of their own services. Member 3 coordinates shared infrastructure and enforces standards.

LAYER 4 – Shared Kubernetes Cluster Governance
Owner: Member 3 · Namespace · Ingress · Monitoring

LAYER 3 – CI/CD Federation
Each member owns their service pipeline
Member 3 reviews for standards compliance

LAYER 2 – Platform / SRE Governance
Member 3: Standards · Shared Infra · Conventions

LAYER 1 – Service Ownership (ALL MEMBERS)
Each member owns: Dockerfile · K8s Manifests · Pipeline

1.2 Anti-Pattern: Centralized DevOps

The following model is explicitly **rejected** for TechHub:

- ✗ ONE member handles all Kubernetes
- ✗ ONE member writes all Dockerfiles
- ✗ ONE member maintains all pipelines

This creates knowledge silos, bottlenecks, burnout, and deployment failures under deadline pressure. The correct model is distributed ownership with central standards.

1.3 Layer 1 — Every Member's DevOps Deliverables

For each service they own, every member must deliver:

- `Dockerfile` — multi-stage, standards-compliant
- Service block in `docker-compose.yml`
- `k8s/<service>/deployment.yaml`
- `k8s/<service>/service.yaml`
- `k8s/<service>/configmap.yaml`
- Health endpoints (`/actuator/health/readiness`, `/actuator/health/liveness`)
- Readiness and liveness probe configuration
- CI/CD workflow file in `.github/workflows/`

1.4 Layer 2 — Member 3 Platform Responsibilities

Standards Governance:

- Docker base image policy and conventions
- Port allocation registry (below)

- Environment variable naming rules
- Kafka topic naming standards
- Kubernetes namespace structure
- CI/CD pipeline naming standards

Shared Infrastructure Management:

- PostgreSQL cluster
 - Redis cluster
 - Apache Kafka + Zookeeper
 - Kubernetes ingress controller
 - Prometheus + Grafana monitoring stack
 - Kubernetes namespace and RBAC
-

2. Shared Responsibility Matrix

2.1 Service Ownership

Service	Code	Docker	K8s	CI/CD
API Gateway	Member 1	Member 1	Member 1	Member 1
User Service	Member 1	Member 1	Member 1	Member 1
Event Service	Member 2	Member 2	Member 2	Member 2
Project Service	Member 2	Member 2	Member 2	Member 2
Team Service	Member 3	Member 3	Member 3	Member 3
Notification Service	Member 3	Member 3	Member 3	Member 3
Community Service	Member 4	Member 4	Member 4	Member 4
Next.js Frontend	Member 4	Member 4	Member 4	Member 4

2.2 Infrastructure Ownership

Component	Primary Owner	Support
PostgreSQL	Member 3	All
Redis	Member 1	Member 3

Component	Primary Owner	Support
Kafka + Zookeeper	Member 3	Member 2
Docker Compose (global file)	Member 3	All contribute
Kubernetes Namespace	Member 3	—
Kubernetes Ingress	Member 3	Member 1
Prometheus	Member 3	—
Grafana Dashboards	Member 4	Member 3
GitHub Actions standards	Member 3	All

2.3 Port Allocation Registry

Service	Internal Port	External Port (Dev)	Owner
API Gateway	8080	8080	Member 1
User Service	8081	8081	Member 1
Event Service	8082	8082	Member 2
Project Service	8083	8083	Member 2
Team Service	8084	8084	Member 3
Community Service	8085	8085	Member 4
Notification Service	8086	8086	Member 3
Next.js Frontend	3000	3000	Member 4
PostgreSQL	5432	5432	Member 3
Redis	6379	6379	Member 1
Kafka	9092	9092	Member 3
Zookeeper	2181	2181	Member 3
Prometheus	9090	9090	Member 3
Grafana	3001	3001	Member 3

3. Docker Standards

3.1 Base Image Policy

Service Type	Required Base Image	Reason
Spring Boot (build stage)	maven:3.9-eclipse-temurin-21	Reproducible builds
Spring Boot (runtime)	eclipse-temurin:21-jre-alpine	Minimal size, security
Next.js (build stage)	node:20-alpine	Minimal, LTS
Next.js (runtime)	node:20-alpine	Consistent

RULES:

- Never use latest tag on base images. Pin to a specific version.
- Always use Alpine variants to minimize image size and attack surface.
- Always use multi-stage builds. Never ship a build environment to production.

3.2 Standard Dockerfile — Spring Boot

dockerfile

```
# — Stage 1: Build —————
FROM maven:3.9-eclipse-temurin-21 AS builder
WORKDIR /app

# Copy pom.xml first — caches dependencies if pom.xml unchanged
COPY pom.xml .
RUN mvn dependency:go-offline -B

COPY src ./src
RUN mvn clean package -DskipTests -B

# — Stage 2: Runtime —————
FROM eclipse-temurin:21-jre-alpine AS runtime
WORKDIR /app

# Security: run as non-root user
RUN addgroup -S techhub && adduser -S techhub -G techhub
USER techhub

COPY --from=builder /app/target/*.jar app.jar

EXPOSE <PORT>

HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
  CMD wget --quiet --tries=1 --spider \
    http://localhost:<PORT>/actuator/health || exit 1

ENTRYPOINT ["java", \
  "-XX:+UseContainerSupport", \
  "-XX:MaxRAMPercentage=75.0", \
  "-Djava.security.egd=file:/dev/./urandom", \
  "-jar", "app.jar"]
```

3.3 Standard Dockerfile — Next.js Frontend

dockerfile

```
# — Stage 1: Dependencies —————
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

# — Stage 2: Build —————
FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

# — Stage 3: Runtime —————
FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production

RUN addgroup -S techhub && adduser -S techhub -G techhub
USER techhub

COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static

EXPOSE 3000

HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
  CMD wget --quiet --tries=1 --spider http://localhost:3000 || exit 1

CMD ["node", "server.js"]
```

3.4 Mandatory `.dockerignore`

Every service directory must contain a `.dockerignore`:

```
# Build output
target/
*.jar

# Source control
.git/
.github/

# Environment files – never ship these
.env
.env.*

# Documentation
*.md
docs/

# IDE artifacts
.idea/
.vscode/
*.iml

# Logs
*.log
logs/

# Node (for frontend)
node_modules/
.next/
```

3.5 Docker Image Tagging Convention

```
techhub/<service-name>:<tag>
```

Tag types:

latest	→ Most recent build from main branch
<git-sha>	→ Immutable, traceable per-commit tag (e.g., a1b2c3d)
<branch-name>	→ Branch-specific tag (e.g., develop)

Examples:

```
techhub/user-service:latest
techhub/user-service:a1b2c3d4e5f6
techhub/event-service:develop
```


3.6 Docker Build Rules Summary

Rule	Requirement
Multi-stage build	Mandatory for all services
Non-root user	Mandatory — create and use a service user
HEALTHCHECK	Mandatory in every Dockerfile
Pin base image version	Never use <code>:latest</code> on base images
<code>.dockerignore</code> present	Mandatory in every service directory
No secrets in image	Mandatory — use env vars at runtime
Single port exposed	Each image exposes exactly one application port

4. Kubernetes Standards

4.1 Namespace

All TechHub services deploy into a single namespace:

```
yaml

# k8s/namespace.yaml — Owner: Member 3
apiVersion: v1
kind: Namespace
metadata:
  name: techhub
  labels:
    project: techhub
    environment: development
    managed-by: member-3
```

4.2 Label Standards

Every Kubernetes resource must carry these labels:

```
yaml
```

```
labels:
  app: <service-name>           # e.g., user-service
  version: "1.0.0"
  team: techhub
  managed-by: <member-id>       # e.g., member-1
```

4.3 Resource Requests and Limits Policy

Tier	CPU Request	CPU Limit	Memory Request	Memory Limit	Use For
Lightweight	125m	250m	128Mi	256Mi	Simple microservices
Standard	250m	500m	256Mi	512Mi	Most Spring Boot services
Stateful	500m	1000m	512Mi	1Gi	PostgreSQL, Kafka, Redis

Deployments without resource requests and limits will be **rejected in code review**.
Unbounded containers cause noisy-neighbor problems and cluster instability.

4.4 Standard Deployment Template

```
yaml
```

```
# k8s/<service-name>/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: <service-name>
  namespace: techhub
  labels:
    app: <service-name>
    version: "1.0.0"
    team: techhub
    managed-by: <member-id>
spec:
  replicas: 1
  selector:
    matchLabels:
      app: <service-name>
  template:
    metadata:
      labels:
        app: <service-name>
        version: "1.0.0"
    spec:
      containers:
        - name: <service-name>
          image: techhub/<service-name>:latest
          imagePullPolicy: Always
          ports:
            - containerPort: <PORT>
              name: http
          envFrom:
            - configMapRef:
                name: <service-name>-config
            - secretRef:
                name: <service-name>-secret
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: <PORT>
            initialDelaySeconds: 20
            periodSeconds: 10
            failureThreshold: 3
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: <PORT>
            initialDelaySeconds: 30
```

```
    periodSeconds: 15
    failureThreshold: 3
  resources:
    requests:
      memory: "256Mi"
      cpu: "250m"
    limits:
      memory: "512Mi"
      cpu: "500m"
```

4.5 Standard Service Template

```
yaml

# k8s/<service-name>/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: <service-name>-svc
  namespace: techhub
  labels:
    app: <service-name>
spec:
  selector:
    app: <service-name>
  ports:
    - name: http
      protocol: TCP
      port: <PORT>
      targetPort: <PORT>
  type: ClusterIP
```

4.6 Standard ConfigMap Template

```
yaml
```

```
# k8s/<service-name>/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: <service-name>-config
  namespace: techhub
data:
  SERVER_PORT: "<PORT>"
  SPRING_APPLICATION_NAME: "<service-name>"
  SPRING_PROFILES_ACTIVE: "k8s"
  DATABASE_HOST: "postgres-svc"
  DATABASE_PORT: "5432"
  DATABASE_NAME: "<service>_db"
  KAFKA_BOOTSTRAP_SERVERS: "kafka-svc:9092"
  REDIS_HOST: "redis-svc"
  REDIS_PORT: "6379"
  MANAGEMENT_ENDPOINTS_WEB_EXPOSURE_INCLUDE: "health,info,metrics,prometheus"
  MANAGEMENT_ENDPOINT_HEALTH_PROBES_ENABLED: "true"
```

4.7 Secret Template

```
yaml

# k8s/<service-name>/secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: <service-name>-secret
  namespace: techhub
type: Opaque
stringData:
  DATABASE_USERNAME: "techhub"
  DATABASE_PASSWORD: "<password>"          # Replace with actual value
  JWT_SECRET: "<jwt-secret>"              # Only for gateway / user-service
```

CRITICAL: Never commit actual secret values to Git. Use `kubectl apply` manually for secrets, or use a secrets manager.

4.8 Probe Configuration Requirements

```
yaml
```

```

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: <PORT>
  initialDelaySeconds: 20    # Give Spring Boot time to start
  periodSeconds: 10
  failureThreshold: 3

livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: <PORT>
  initialDelaySeconds: 30    # Longer delay – process must be fully up
  periodSeconds: 15
  failureThreshold: 3

```

Probes are **mandatory** in every service Deployment. Missing probes = Definition of Done failure.

5. Kubernetes Ownership Matrix

Kubernetes Resource	Owner	Support
Namespace <code>techhub</code>	Member 3	—
Ingress Controller	Member 3	Member 1
API Gateway Deployment	Member 1	—
User Service Deployment	Member 1	—
Event Service Deployment	Member 2	—
Project Service Deployment	Member 2	—
Team Service Deployment	Member 3	—
Notification Service Deployment	Member 3	—
Community Service Deployment	Member 4	—
Kafka StatefulSet	Member 3	—
Zookeeper StatefulSet	Member 3	—
PostgreSQL StatefulSet	Member 3	—

Kubernetes Resource	Owner	Support
Redis Deployment	Member 1	Member 3
Prometheus Deployment	Member 3	—
Grafana Deployment	Member 3	Member 4
PersistentVolumeClaims	Member 3	—
ConfigMaps (own services)	Each member	—
Secrets (own services)	Each member	—

6. Kubernetes Folder Structure

```
k8s/
|
├─ namespace.yaml           ← Member 3
├─ ingress.yaml            ← Member 3
|
├─ gateway/                ← Member 1
|   ├─ deployment.yaml
|   ├─ service.yaml
|   └─ configmap.yaml
|
├─ user-service/           ← Member 1
|   ├─ deployment.yaml
|   ├─ service.yaml
|   ├─ configmap.yaml
|   └─ secret.yaml
|
├─ event-service/          ← Member 2
|   ├─ deployment.yaml
|   ├─ service.yaml
|   └─ configmap.yaml
|
├─ project-service/        ← Member 2
|   ├─ deployment.yaml
|   ├─ service.yaml
|   └─ configmap.yaml
|
├─ team-service/           ← Member 3
|   ├─ deployment.yaml
|   └─ service.yaml
```

```
|   └─ configmap.yaml
|
└─ notification-service/           ← Member 3
    │ └─ deployment.yaml
    │ └─ service.yaml
    └─ configmap.yaml
|
└─ community-service/             ← Member 4
    │ └─ deployment.yaml
    │ └─ service.yaml
    └─ configmap.yaml
|
└─ frontend/                      ← Member 4
    │ └─ deployment.yaml
    │ └─ service.yaml
    └─ configmap.yaml
|
└─ infrastructure/                ← Member 3
    │
    │ └─ kafka/
    │     │ └─ statefulset.yaml
    │     │ └─ service.yaml
    │     └─ configmap.yaml
    │
    │ └─ zookeeper/
    │     │ └─ statefulset.yaml
    │     └─ service.yaml
    │
    │ └─ postgres/
    │     │ └─ statefulset.yaml
    │     │ └─ service.yaml
    │     └─ pvc.yaml
    │
    │ └─ redis/
    │     │ └─ deployment.yaml
    │     └─ service.yaml
    │
    └─ monitoring/
        │ └─ prometheus/
        │     │ └─ deployment.yaml
        │     │ └─ configmap.yaml      ← scrape config
        │     └─ service.yaml
        └─ grafana/
            │ └─ deployment.yaml
            └─ service.yaml
```


7. Docker Compose Integration Plan

7.1 Compose File Structure

yaml

```
# docker-compose.yml – Coordinated by Member 3
# Each member is responsible for their own service block.

version: '3.9'

networks:
  techhub-network:
    driver: bridge

volumes:
  postgres-data:
  redis-data:
  kafka-data:

services:

  # — Infrastructure — Owner: Member 3 —————

  postgres:
    image: postgres:15-alpine
    container_name: techhub-postgres
    environment:
      POSTGRES_USER: techhub
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_MULTIPLE_DATABASES: "user_db,event_db,project_db,team_db,community_db"
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data
      - ./scripts/init-db.sh:/docker-entrypoint-initdb.d/init-db.sh
    networks:
      - techhub-network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U techhub"]
      interval: 10s
      timeout: 5s
      retries: 5

  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    container_name: techhub-zookeeper
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    networks:
      - techhub-network
```

```
kafka:
  image: confluentinc/cp-kafka:7.5.0
  container_name: techhub-kafka
  depends_on:
    - zookeeper
  ports:
    - "9092:9092"
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAIN
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
  networks:
    - techhub-network
```

```
redis:
  image: redis:7-alpine
  container_name: techhub-redis
  ports:
    - "6379:6379"
  volumes:
    - redis-data:/data
  command: redis-server --save 60 1 --loglevel warning
  networks:
    - techhub-network
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
    timeout: 5s
    retries: 3
```

— Member 1: API Gateway —————

```
api-gateway:
  build:
    context: ./gateway
    dockerfile: Dockerfile
  container_name: techhub-api-gateway
  ports:
    - "8080:8080"
  environment:
    SPRING_PROFILES_ACTIVE: docker
    USER_SERVICE_URL: http://user-service:8081
    EVENT_SERVICE_URL: http://event-service:8082
```

```
PROJECT_SERVICE_URL: http://project-service:8083
TEAM_SERVICE_URL: http://team-service:8084
COMMUNITY_SERVICE_URL: http://community-service:8085
NOTIFICATION_SERVICE_URL: http://notification-service:8086
REDIS_HOST: redis
JWT_SECRET: ${JWT_SECRET}
depends_on:
  redis:
    condition: service_healthy
networks:
  - techhub-network

# — Member 1: User Service —————

user-service:
  build:
    context: ./user-service
    dockerfile: Dockerfile
  container_name: techhub-user-service
  ports:
    - "8081:8081"
  environment:
    SPRING_PROFILES_ACTIVE: docker
    DATABASE_HOST: postgres
    DATABASE_PORT: 5432
    DATABASE_NAME: user_db
    DATABASE_USERNAME: techhub
    DATABASE_PASSWORD: ${DB_PASSWORD}
    KAFKA_BOOTSTRAP_SERVERS: kafka:9092
    REDIS_HOST: redis
    JWT_SECRET: ${JWT_SECRET}
  depends_on:
    postgres:
      condition: service_healthy
    kafka:
      condition: service_started
  networks:
    - techhub-network

# — Member 2: Event Service —————
# event-service: [same pattern as above]

# — Member 2: Project Service —————
# project-service: [same pattern]

# — Member 3: Team Service —————
# team-service: [same pattern]
```

```
# — Member 3: Notification Service —
# notification-service: [same pattern]

# — Member 4: Community Service —
# community-service: [same pattern]

# — Member 4: Frontend —
# frontend: [same pattern, port 3000]
```

7.2 Required `.env` File

Create a `.env` file at the root (never commit this):

```
bash

# .env - Copy from .env.example, fill in values locally
DB_PASSWORD=your_local_password
JWT_SECRET=your_jwt_secret_at_least_32_chars
```

7.3 `.env.example` (Commit This to Git)

```
bash

# .env.example - Safe to commit. Shows required variables without values.
DB_PASSWORD=
JWT_SECRET=
```

7.4 Docker Compose Integration Timeline

Target Day	Milestone
Day 1	Infrastructure running: <code>(docker compose up postgres redis kafka zookeeper -d)</code>
Day 3	Gateway + User Service added, health checks pass
Day 5	Event Service + Project Service added, Kafka flow verified
Day 7	Team Service + Notification Service added, event notifications working
Day 9	Community Service added, full mesh tested
Day 10	Frontend added, <code>(docker compose up)</code> starts entire stack
Day 12	All health checks green, integration tests pass against Compose

7.5 Useful Docker Compose Commands

```
bash

# Start all services (detached)
docker compose up -d

# Start only infrastructure
docker compose up postgres redis kafka zookeeper -d

# View logs for a specific service
docker compose logs -f user-service

# Rebuild and restart a service
docker compose up --build user-service -d

# Stop all services
docker compose down

# Stop and delete volumes (full reset)
docker compose down -v

# Check health of all services
docker compose ps
```

8. Kubernetes Progressive Deployment Plan

8.1 Deployment Phases

Deploy services incrementally. Never wait for all services to be complete before starting Kubernetes.

PHASE 0 – Cluster & Shared Infrastructure (Member 3)

Target: Day 5

- └─ kubectl apply -f k8s/namespace.yaml
- └─ kubectl apply -f k8s/infrastructure/postgres/
- └─ kubectl apply -f k8s/infrastructure/zookeeper/
- └─ kubectl apply -f k8s/infrastructure/kafka/
- └─ kubectl apply -f k8s/infrastructure/redis/
- └─ Validate: All infrastructure pods Running/1/1

PHASE 1 – Gateway + User Service (Member 1)

Target: Day 6

- └─ kubectl apply -f k8s/gateway/
- └─ kubectl apply -f k8s/user-service/
- └─ Validate: HTTP request through gateway to user service

PHASE 2 – Event + Project Services (Member 2)

Target: Day 7

- └─ kubectl apply -f k8s/event-service/
- └─ kubectl apply -f k8s/project-service/
- └─ Validate: Kafka events produced and consumed

PHASE 3 – Team + Notification Services (Member 3)

Target: Day 8

- └─ kubectl apply -f k8s/team-service/
- └─ kubectl apply -f k8s/notification-service/
- └─ Validate: Notification events triggered correctly

PHASE 4 – Community Service (Member 4)

Target: Day 9

- └─ kubectl apply -f k8s/community-service/
- └─ Validate: Full service mesh communication

PHASE 5 – Ingress + Monitoring (Member 3)

Target: Day 10

- └─ kubectl apply -f k8s/ingress.yaml
- └─ kubectl apply -f k8s/infrastructure/monitoring/
- └─ Validate: External access · Dashboards visible

PHASE 6 – Frontend (Member 4)

Target: Day 11

- └─ kubectl apply -f k8s/frontend/
- └─ Validate: End-to-end user flow through browser

8.2 Validation After Each Phase

After every phase, run:

```
bash
```

```
# All pods must show STATUS=Running, READY=1/1
```

```
kubectl get pods -n techhub
```

```
# All services must be present
```

```
kubectl get services -n techhub
```

```
# Zero pods in: CrashLoopBackOff, ImagePullBackOff, Pending
```

8.3 Common Kubernetes Debugging Commands

```
bash
```

```
# View pod logs
```

```
kubectl logs -f deployment/<service-name> -n techhub
```

```
# Describe a pod (events, probe failures, resource issues)
```

```
kubectl describe pod <pod-name> -n techhub
```

```
# Port-forward for local debugging (without ingress)
```

```
kubectl port-forward svc/<service-name>-svc <local-port>:<port> -n techhub
```

```
# Execute a shell in a running pod
```

```
kubectl exec -it deployment/<service-name> -n techhub -- /bin/sh
```

```
# View events (helpful for startup failures)
```

```
kubectl get events -n techhub --sort-by='.lastTimestamp'
```

```
# Roll back a deployment
```

```
kubectl rollout undo deployment/<service-name> -n techhub
```

```
# View rollout history
```

```
kubectl rollout history deployment/<service-name> -n techhub
```

9. Environment Variable Standards

9.1 Naming Convention

```
<CATEGORY>_<SUBCATEGORY>_<VARIABLE>
```

Examples:

```
DATABASE_HOST
```

```
DATABASE_PORT
```



```
DATABASE_NAME
DATABASE_USERNAME
DATABASE_PASSWORD
KAFKA_BOOTSTRAP_SERVERS
KAFKA_GROUP_ID
REDIS_HOST
REDIS_PORT
JWT_SECRET
JWT_EXPIRATION_MS
SERVER_PORT
SPRING_APPLICATION_NAME
SPRING_PROFILES_ACTIVE
```

9.2 Standard Variables for All Services

```
properties

# Server
SERVER_PORT=<port>
SPRING_APPLICATION_NAME=<service-name>
SPRING_PROFILES_ACTIVE=docker    # or: k8s

# Database
DATABASE_HOST=postgres           # Service name in Docker/K8s
DATABASE_PORT=5432
DATABASE_NAME=<service>_db
DATABASE_USERNAME=techhub
DATABASE_PASSWORD=${DB_PASSWORD} # Always from Secret

# Kafka
KAFKA_BOOTSTRAP_SERVERS=kafka:9092
KAFKA_GROUP_ID=<service-name>-group

# Observability
MANAGEMENT_ENDPOINTS_WEB_EXPOSURE_INCLUDE=health,info,metrics,prometheus
MANAGEMENT_ENDPOINT_HEALTH_PROBES_ENABLED=true
MANAGEMENT_ENDPOINT_HEALTH_SHOW_DETAILS=always
```

9.3 Secrets Policy

Variable Type	Storage
Passwords (DB, JWT, API keys)	Kubernetes Secret
Non-sensitive configuration	Kubernetes ConfigMap
Public service URLs	Kubernetes ConfigMap

10. Service Naming Conventions

10.1 Kubernetes Resource Names

Resource Type → Naming Pattern → Example		
Deployment	→ <service-name>	→ user-service
Service	→ <service-name>-svc	→ user-service-svc
ConfigMap	→ <service-name>-config	→ user-service-config
Secret	→ <service-name>-secret	→ user-service-secret

10.2 Full Name Registry

Service	Spring App Name	Docker Image	K8s Deployment	K8s Service
API Gateway	api-gateway	techhub/api-gateway	api-gateway	api-gateway-svc
User Service	user-service	techhub/user-service	user-service	user-service-svc
Event Service	event-service	techhub/event-service	event-service	event-service-svc
Project Service	project-service	techhub/project-service	project-service	project-service-svc
Team Service	team-service	techhub/team-service	team-service	team-service-svc
Notification Service	notification-service	techhub/notification-service	notification-service	notification-service-svc
Community Service	community-service	techhub/community-service	community-service	community-service-svc

10.3 Database Schema Names

Each service owns its own PostgreSQL database schema:

Service	Database Name
User Service	user_db
Event Service	event_db
Project Service	project_db
Team Service	team_db
Notification Service	notification_db
Community Service	community_db

Services must NEVER access another service's database directly. Always go through the service's API or Kafka events.

11. Local Development Strategy

11.1 Prerequisites

Tool	Version	Purpose
Java JDK	21	Spring Boot development
Maven	3.9.x	Build tool
Docker Desktop	24.x	Container runtime + local Kubernetes
kubectl	1.28.x	Kubernetes CLI
Minikube or Kind	Latest	Local Kubernetes cluster
IntelliJ IDEA	Latest	IDE
Node.js	20.x LTS	Frontend development

11.2 Local Development Flow

```
bash
```

```
# 1. Clone the repository
git clone https://github.com/<org>/techhub.git
cd techhub

# 2. Set up environment variables
cp .env.example .env
# Edit .env with your local values

# 3. Start shared infrastructure
docker compose up postgres redis kafka zookeeper -d

# 4. Run your service(s) locally (IDE or CLI)
cd user-service
mvn spring-boot:run

# OR: run everything in Docker
docker compose up --build
```

11.3 Spring Boot Application Profiles

Profile	How Activated	Use Case
default	No profile	Local IDE development (use localhost)
docker	SPRING_PROFILES_ACTIVE=docker	Docker Compose (use service names)
k8s	SPRING_PROFILES_ACTIVE=k8s	Kubernetes (JSON logs, K8s-aware config)

12. Deployment Workflow

12.1 Standard Flow

```
Push to feature branch
  |
  ▼
GitHub Actions: Build + Test
  |
  ▼
Create Pull Request → Code Review
  |
  ▼
Merge to develop
  |
```

▼
GitHub Actions: Docker Build + Push
|
▼
kubectl apply -f k8s/<service>/
|
▼
Kubernetes applies Deployment
|
▼
Readiness probe confirms pod healthy
|
▼
Integration verified

12.2 Rollback Procedure

```
bash

# Rollback a service to previous version
kubectl rollout undo deployment/<service-name> -n techhub

# Confirm rollback succeeded
kubectl rollout status deployment/<service-name> -n techhub

# Rollback to a specific revision
kubectl rollout undo deployment/<service-name> --to-revision=2 -n techhub
```

13. Redis Caching Standards

13.1 Ownership

Redis is owned and operated by **Member 1**. All other services requiring Redis caching must coordinate with Member 1 on key naming and TTL configuration.

13.2 Cache Key Naming

```
techhub:<service>:<entity>:<identifier>
```

Examples:

```
techhub:user:profile:usr-001
techhub:event:details:evt-123
techhub:project:list:page-1
techhub:team:members:team-456
```

13.3 TTL Policy

Data Type	TTL
User profile	30 minutes
Event details	15 minutes
Project list	10 minutes
Team members	20 minutes
Auth session token	1 hour
Static reference data	24 hours

13.4 Spring Boot Redis Configuration

```
properties

spring.data.redis.host=${REDIS_HOST:localhost}
spring.data.redis.port=${REDIS_PORT:6379}
spring.cache.type=redis
spring.cache.redis.time-to-live=1800000      # 30 min in ms
spring.cache.redis.cache-null-values=false
spring.cache.redis.key-prefix=techhub:
```

14. Health Check Standards

14.1 Required Endpoints

Endpoint	Purpose	Required
/actuator/health	Overall status	✔ Mandatory
/actuator/health/readiness	K8s readiness probe	✔ Mandatory
/actuator/health/liveness	K8s liveness probe	✔ Mandatory
/actuator/info	Service metadata	✔ Mandatory
/actuator/metrics	JVM and app metrics	✔ Mandatory
/actuator/prometheus	Prometheus scrape endpoint	✔ Mandatory

14.2 Required Spring Boot Configuration

properties

```
# application.properties – all services must include this
management.endpoints.web.exposure.include=health,info,metrics,prometheus
management.endpoint.health.show-details=always
management.endpoint.health.probes.enabled=true
management.health.kafka.enabled=true
management.health.db.enabled=true
management.health.redis.enabled=true
management.metrics.export.prometheus.enabled=true
```

14.3 Expected Health Response

json

```
{
  "status": "UP",
  "components": {
    "db": { "status": "UP" },
    "kafka": { "status": "UP" },
    "redis": { "status": "UP" },
    "livenessState": { "status": "UP" },
    "readinessState": { "status": "UP" }
  }
}
```